

Capítulo 5: Camada de Rede, Plano de Controle

21 de junho de 2026

Esse documento cobre o Capítulo 5 (Plano de Controle). Antes de escrever, vou conferir o conteúdo real dos seus slides da 9ª edição para o resumo bater certinho com o que cai na sua Prova 3. Bom material. Vou conferir vetor de distâncias, OSPF, BGP e SDN para garantir alinhamento. Excelente — tenho o vetor de distâncias completo, com o exemplo de count-to-infinity. Última verificação: OSPF, BGP, SDN e ICMP. Conferido com seus slides da 9ª edição (e com os materiais de Packet Tracer). Está tudo alinhado.

Mapa do dia

A pergunta-mãe do capítulo é: “**quem decide os caminhos que os pacotes percorrem na rede?**” O Cap. 4 (plano de dados) respondeu *como* um pacote é jogado da porta de entrada para a de saída. Agora o foco é *como a tabela de encaminhamento é calculada*.

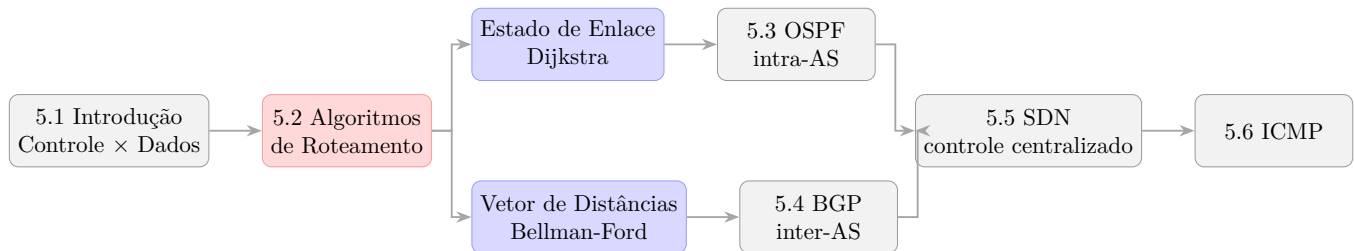


Figura 1: Dependência lógica e fluxo de tópicos do Capítulo 5.

A dependência lógica é essa: você precisa de **estado de enlace** para entender OSPF, e da intuição de **vetor de distâncias** para entender BGP. Por isso o capítulo é estudado em bloco.

1 5.1 — Roteamento × Encaminhamento (a distinção que cai sempre)

Essa diferença apareceu literalmente na Q10 do seu gabarito anterior. Memorize a dicotomia **macro** × **micro**:

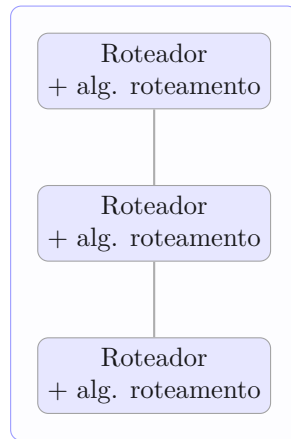
Tabela 1: Comparação entre Roteamento e Encaminhamento

	Roteamento (Routing)	Encaminhamento (Forwarding)
Plano	Controle	Dados
Escala	Rede inteira (fim-a-fim)	Local (um roteador)
Velocidade	Lento (segundos/ms)	Rápido (nanossegundos)
O que faz	Calcula os melhores caminhos e constrói a tabela	Consulta a tabela e move o pacote da porta de entrada → saída
Implementação	Software (CPU do roteador / controlador)	Hardware (ASIC)

Reflexão Crítica

Roteamento é “planejar a viagem com o mapa”; encaminhamento é “virar à direita no cruzamento”. A separação importa porque é exatamente o que o **SDN** explora — ele *arranca* o roteamento de dentro do roteador e o centraliza.

Tradicional: controle por roteador



SDN: controle logicamente centralizado

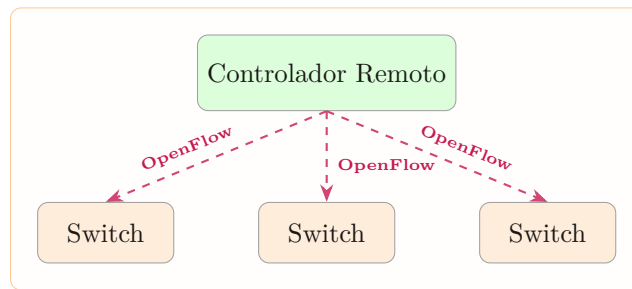


Figura 2: Abordagem tradicional com controle distribuído vs. Abordagem baseada em SDN.

Duas abordagens de implementação do plano de controle:

1.1 A Separação dos Planos de Dados e Controle: Conceito e Motivação

Historicamente, nas redes de computadores tradicionais, o plano de dados e o plano de controle vinham integrados de forma monolítica dentro de cada roteador individual (uma solução física fechada do fabricante). No entanto, o paradigma moderno de redes (especialmente impulsionado pelo SDN) consolidou uma separação explícita entre esses dois planos.

1.1.1 Conceito

A distinção fundamental reside no escopo de atuação de cada um dos planos:

- **Plano de Dados (Local):** O plano de dados atua em cada roteador individualmente. Sua principal função é o *encaminhamento* (forwarding): pegar um datagrama recebido em uma interface de entrada, inspecionar seu cabeçalho e direcioná-lo para a interface de saída correspondente, com base na tabela de encaminhamento local. É executado em hardware especializado (ASICs) para garantir tempos de processamento da ordem de nanossegundos.
- **Plano de Controle (Global):** O plano de controle coordena a rede fim-a-fim. Sua principal função é o *roteamento* (routing): determinar a trajetória (caminho completo) que um pacote fará desde o host de origem até o host de destino. Ele calcula os valores que preenchem as tabelas de encaminhamento do plano de dados. É implementado em software na CPU e processado em milissegundos.

1.1.2 Por que houve a divisão?

A decisão de separar o plano de controle do plano de dados decorre de limitações fundamentais da arquitetura tradicional integrada:

1. **Complexidade de Gerenciamento:** Redes tradicionais dependem de algoritmos distribuídos e assíncronos rodando de forma independente em centenas ou milhares de roteadores. A gerência desse sistema distribuído era complexa e propensa a falhas humanas de configuração (*misconfigurations*).
2. **Engenharia de Tráfego Inflexível:** No roteamento tradicional, os pacotes são encaminhados exclusivamente com base no endereço IP de destino (*destination-based routing*). Se um administrador deseja desviar fluxos de tráfego específicos (como separar tráfego de voz de dados) por caminhos diferentes que possuem a mesma origem e destino, isso era extremamente complexo e ineficiente de programar em protocolos clássicos de roteamento.
3. **Falta de Inovação (Vendor Lock-in):** Como o plano de controle rodava em sistemas operacionais proprietários e fechados (como o Cisco IOS) dentro de caixas pretas de hardware, propor novos algoritmos ou protocolos exigia o aval e atualização de software por parte das próprias fabricantes. A separação permite que o plano de controle seja escrito em software livre e rodado em servidores comerciais genéricos.

Ao desacoplar o plano de controle do plano de dados, o SDN tornou as redes programáveis. Os roteadores tornaram-se switches generalizados e simplificados de Match+Action (plano de dados), e toda a lógica de roteamento foi concentrada de forma centralizada e global no controlador SDN (plano de controle).

2 5.2 — Algoritmos de roteamento (o núcleo do dia)

2.1 Abstração de grafo

A rede vira um grafo $G = (N, E)$, onde N são os roteadores e E os enlaces. Cada enlace tem um custo $c_{a,b}$ definido pelo operador — pode ser sempre 1 (conta saltos), ou inversamente proporcional à banda, ou ao congestionamento. Se não há enlace direto, $c_{a,b} = \infty$.

2.2 Classificação (cai em V/F — Q1)

Tabela 2: Classificação dos Algoritmos de Roteamento

Critério	Tipo 1	Tipo 2
Informação	Global (todos conhecem a topologia toda) → <i>estado de enlace</i>	Descentralizado (cada nó só conhece vizinhos) → <i>vetor de distâncias</i>
Dinâmica	Estático (rotas mudam devagar)	Dinâmico (mudam rápido, por updates periódicos ou eventos)

2.3 A) Estado de Enlace — Algoritmo de Dijkstra

Ideia central: cada roteador faz um *link-state broadcast* (inunda a rede com o custo de seus enlaces). Depois disso, **todos têm a topologia completa** e cada um roda Dijkstra localmente para achar os caminhos de menor custo a partir de si.

Notação:

- $D(v)$: estimativa atual do custo do caminho mínimo da origem até v .
- $p(v)$: nó predecessor de v no caminho.
- N' : conjunto de nós cujo caminho mínimo já é **definitivo**.

O laço de Dijkstra:

$$D(v) = \min(D(v), D(w) + c_{w,v})$$

A cada iteração, escolhe-se o nó $w \notin N'$ de menor $D(w)$, adiciona-se a N' , e relaxam-se seus vizinhos.

2.3.1 Exemplo passo a passo (o grafo dos seus slides)

Origem = u . Grafo com custos:

Tabela 3: Tabela de iterações do Dijkstra a partir da origem u

Step	N'	$D(v), p$	$D(w), p$	$D(x), p$	$D(y), p$	$D(z), p$
0	u	$2, u$	$5, u$	1, u	∞	∞
1	ux	$2, u$	$4, x$		2, x	∞
2	uxy	2, u	$3, y$			$4, y$
3	$uxyv$		3, y			$4, y$
4	$uxyvw$					4, y
5	$uxyvwz$					

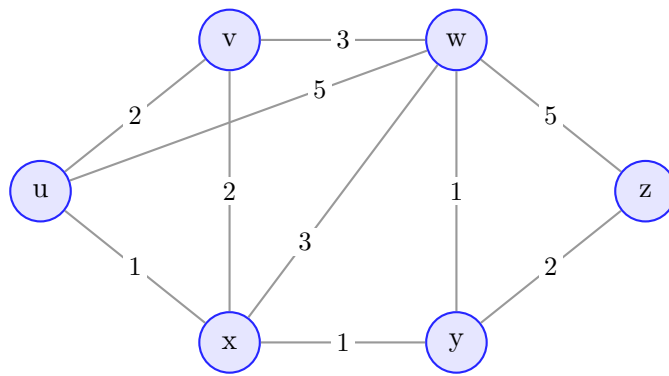


Figura 3: Grafo de exemplo para execução do Algoritmo de Dijkstra.

Como ler (o examinador quer ver a derivação, não só o resultado):

- **Step 0:** inicializa com custos diretos. Menor fora de N' : x (custo 1) $\rightarrow$ entra.
- **Step 1:** relaxa vizinhos de x . $D(w) = \min(5, 1+3) = 4$; $D(y) = \min(\infty, 1+1) = 2$. Empate entre $v(2)$ e $y(2) \rightarrow$ desempate arbitrário, escolhe y .
- **Step 2:** relaxa vizinhos de y . $D(w) = \min(4, 2+1) = 3$ (via y); $D(z) = \min(\infty, 2+2) = 4$. Menor: $v(2) \rightarrow$ entra.
- **Step 3–4:** $w(3)$ e depois $z(4)$ entram. Note que $D(z)$ ficou em 4 via y , pois $3+5 = 8 > 4$.

Tabela de encaminhamento resultante em u (extraída da árvore de menor custo):

Tabela 4: Tabela de encaminhamento resultante no nó u

Destino	Enlace de saída	Custo
v	(u, v) direto	2
x	(u, x) direto	1
y	(u, x)	2
w	(u, x)	3
z	(u, x)	4

Nota / Atenção

Repare: *todos os destinos menos v saem por (u, x)* . O enlace barato $u-x$ vira a “porta de entrada” da rede para u .

Complexidade e limitações:

- n iterações, cada uma checando até n nós $\rightarrow O(n^2)$ (melhorável para $O(n \log n)$ com heap).
- Mensagens: cada roteador faz broadcast $\rightarrow$ complexidade de mensagens $O(n^2)$.
- **Oscilações:** se o custo depende do volume de tráfego, as rotas podem oscilar (todos migram para o enlace “barato”, que fica congestionado, e o ciclo se repete). É um ponto crítico de prova.

2.4 B) Vetor de Distâncias — Equação de Bellman-Ford

Ideia central: ninguém conhece a topologia toda. Cada nó só sabe o custo aos vizinhos e **troca seu vetor de distâncias** periodicamente com eles. A informação se difunde iterativamente pela rede.

Equação de Bellman-Ford (programação dinâmica):

$$D_x(y) = \min_v \{ c_{x,v} + D_v(y) \}$$

onde o mínimo é tomado sobre **todos os vizinhos v de x** . O vizinho que atinge o mínimo é o **próximo salto** no caminho.

2.4.1 Exemplo (dos seus slides)

u quer alcançar z . Os vizinhos v, x, w já anunciaram: $D_v(z) = 5$, $D_x(z) = 3$, $D_w(z) = 3$.

$$D_u(z) = \min \begin{cases} c_{u,v} + D_v(z) = 2 + 5 = 7 \\ c_{u,x} + D_x(z) = 1 + 3 = 4 \\ c_{u,w} + D_w(z) = 5 + 3 = 8 \end{cases} = 4 \quad (\text{próximo salto: } x)$$

Características operacionais:

- **Iterativo e assíncrono:** cada nó recalcula quando muda um custo local *ou* chega um vetor de vizinho.
- **Distribuído e auto-terminável:** o nó só avisa os vizinhos **se** seu vetor mudou. Sem mudança $\rightarrow$ sem mensagem.

2.4.2 O problema do count-to-infinity (cai em prova!)

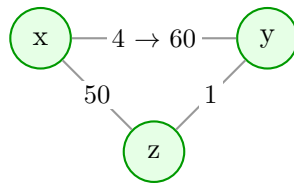


Figura 4: Exemplo do count-to-infinity com o enlace $x - y$ encarecendo.

- **“Good news travels fast”:** se um enlace *barateia*, a boa notícia se propaga rápido.
- **“Bad news travels slow”:** se o enlace $y-x$ *encarece* de 4 para 60, surge um laço de roteamento:
 - y vê custo direto 60, mas z anunciou caminho de custo 5 $\rightarrow y$ calcula “6 via z ”.
 - z aprende “7 via y ” $\rightarrow y$ aprende “8 via z ” $\rightarrow \dots$ sobe de 1 em 1 até estabilizar. Lentíssimo.

Reflexão Crítica

O count-to-infinity existe porque z não sabe que seu “caminho barato para x ” na verdade *passava por* y . Soluções como *poisoned reverse* e *split horizon* mitigam, mas não resolvem todos os casos. Isso é uma vantagem estrutural do estado de enlace: lá cada nó tem a topologia completa, então não cria essas ilusões.

2.5 Comparação LS $\times$ DV (formato clássico das suas provas)

Tabela 5: Comparação entre Estado de Enlace (LS) e Vetor de Distâncias (DV)

Aspecto	Estado de Enlace (LS)	Vetor de Distâncias (DV)
Informação	Global (topologia completa)	Local (só vizinhos)
Algoritmo	Dijkstra	Bellman-Ford
Mensagens	$O(n^2)$, broadcast a todos	Só entre vizinhos
Convergência	$O(n^2)$; pode oscilar	Variável; laços e count-to-infinity
Robustez	Anuncia custo errado $\rightarrow$ afeta só o próprio cálculo de cada nó	Anuncia caminho errado $\rightarrow$ propaga pela rede (<i>black-holing</i>)
Protocolo real	OSPF	RIP , EIGRP

3 5.3 — OSPF (Open Shortest Path First): roteamento intra-AS

Primeiro, a motivação de **escala**: a Internet tem bilhões de destinos. Não dá para um roteador guardar todos, nem para todos rodarem um único algoritmo “plano”. A solução é agregar roteadores em **Sistemas Autônomos (AS)** — e separar:

- **Intra-AS** (dentro de um AS): OSPF, RIP, EIGRP.
- **Inter-AS** (entre ASes): BGP.

OSPF é estado de enlace clássico:

- “Open” = especificação pública ($\neq$ proprietário).
- Cada roteador inunda *link-state advertisements* para todo o AS — **direto sobre IP** (protocolo 89), sem TCP/UDP.
- Cada um monta a topologia completa e roda **Dijkstra**.
- Métricas configuráveis: banda, atraso (não apenas saltos).
- **Mensagens autenticadas** (segurança contra intrusos).

3.1 OSPF hierárquico

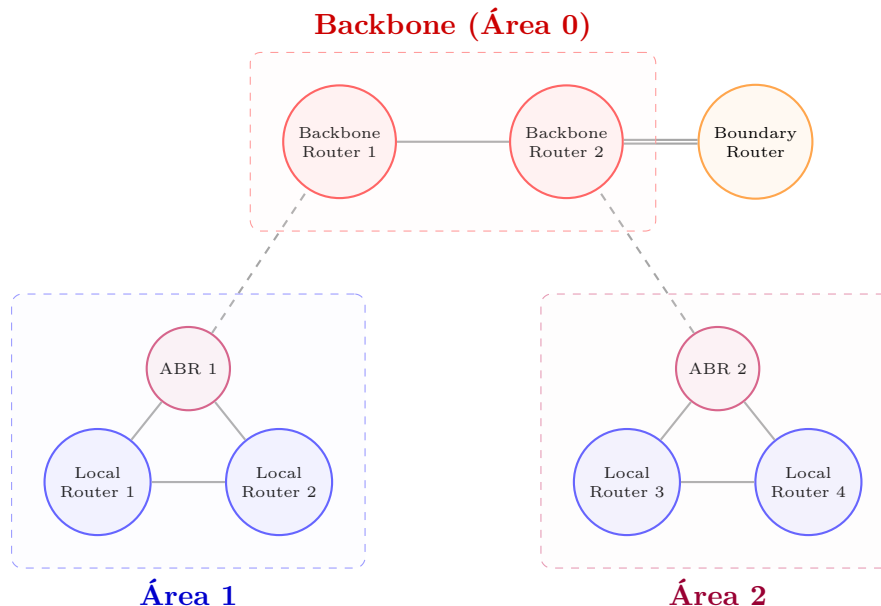


Figura 5: Arquitetura hierárquica do OSPF dividido em áreas.

- **Dois níveis**: área local + backbone (Área 0).
- LSAs inundam **só dentro da área**; *area border routers* (ABR) resumizam distâncias e anunciam no backbone.
- **Todo tráfego entre áreas passa pela Área 0**. Áreas não-backbone não trocam tráfego diretamente.
- *Por que dividir?* Numa rede grande, manter a base de estado de enlace idêntica e atualizada em todos os roteadores fica inviável → a hierarquia reduz tabela e tráfego de controle.

Aplicação Prática

Na config Cisco, `router ospf 1` define o process-id e `network 192.168.1.0 0.0.0.255 area 0` associa redes à área 0 — exatamente a hierarquia acima. RIP × OSPF na Cisco: RIP usa saltos e serve redes pequenas; OSPF usa Dijkstra + banda e escala para redes grandes.

4 5.4 — BGP (Border Gateway Protocol): roteamento inter-AS

O BGP é a “cola que mantém a Internet unida” — o protocolo de roteamento inter-domínio de fato. Ele permite que um AS anuncie: “*estou aqui, alcanço estes destinos, e por este caminho.*”

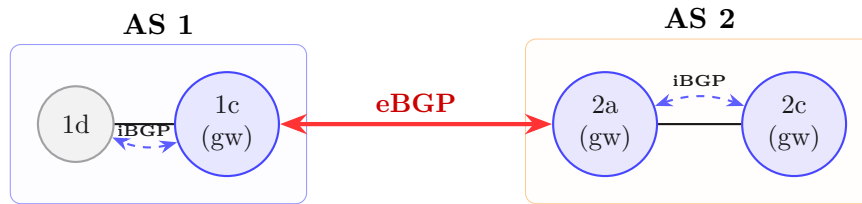


Figura 6: Diferença na propagação BGP interna (iBGP) e externa (eBGP).

Duas variantes:

- **eBGP**: obtém info de alcançabilidade de ASes vizinhos (entre gateways de ASes distintos).
- **iBGP**: propaga essa info para todos os roteadores *internos* do AS.

Anúncio de caminhos (path vector): o BGP não anuncia só custo — anuncia o **AS-PATH** (a sequência de ASes). Quando AS3 anuncia AS3,X para AS2, ele *promete encaminhar* datagramas para X. AS2 propaga AS2,AS3,X, e assim por diante. O AS-PATH evita laços (se um AS se vê no caminho, rejeita).

Seleção de rota (critérios em ordem):

1. **Local preference** (decisão de política / negócio).
2. Menor **AS-PATH**.
3. **NEXT-HOP mais próximo** → *hot potato routing*.
4. Critérios adicionais.

Hot potato routing: o AS escolhe o gateway de saída de **menor custo intra-domínio** (joga a “batata quente” para fora o mais rápido possível), **ignorando** o custo inter-domínio.

Reflexão Crítica

Dentro do AS, há um único administrador → o foco é **performance**. Entre ASes, há interesses comerciais conflitantes → a **política domina**. É por isso que um provedor pode escolher *não* anunciar uma rota mesmo que ela exista: ele não quer carregar tráfego de trânsito de graça entre outros ISPs.

5 5.5 — SDN (Software-Defined Networking)

Contexto histórico: até ≈ 2005 , o plano de controle era **distribuído e por roteador** — cada roteador monolítico rodava implementações proprietárias (IP, OSPF, BGP) num SO fechado (Cisco IOS), com *middleboxes* separadas (firewall, NAT, load balancer). O SDN propõe **arrancar o controle do hardware** e centralizá-lo logicamente.

As 3 características do SDN (cai direto em V/F — Q1):

Como o controlador instala rotas (interação controle/dados):

Tabela 6: As Três Características do SDN

Característica	O que significa
1. Plano de controle separado do de dados	Switches “burros” só encaminham; a inteligência sai deles.
2. Controle logicamente centralizado	Um <i>controlador SDN</i> (software) computa as tabelas para todos.
3. Encaminhamento generalizado (match+action)	Regras baseadas em campos de enlace/rede/transporte — não apenas IP de destino.

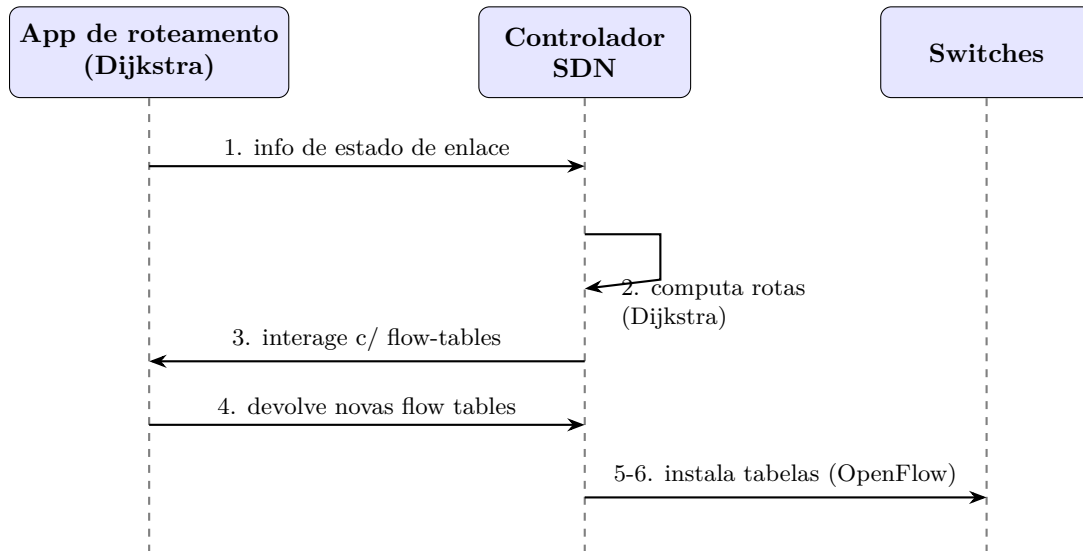


Figura 7: Fluxo de mensagens na instalação de rotas por meio do controlador SDN.

Por que centralizar?

- Gerência mais fácil (evita desconfigurações, dá flexibilidade aos fluxos).
- “Programar” a rede centralmente é mais simples que via algoritmo distribuído.
- Implementação aberta → fomenta inovação.

Analogia

É a revolução *mainframe* → *PC*. Antes, hardware + SO + apps verticalmente integrados e fechados (inovação lenta). Depois, interfaces abertas e horizontais (microprocessador, SO, apps separados) → indústria gigante e inovação rápida. **Engenharia de tráfego** ilustra bem: forçar tráfego $u \rightarrow z$ pela rota $uvwz$ em vez de xyz é *difícil* no roteamento tradicional (você teria que mexer em pesos de enlace e torcer), mas trivial no SDN (você simplesmente programa a flow table).

6 5.6 — ICMP (Internet Control Message Protocol)

Protocolo de **mensagens de controle e erro** entre hosts/roteadores. Carregado *dentro* de datagramas IP. Tem campos **type** e **code**. Os principais para prova:

Traceroute usa ICMP de forma engenhosa:

- A origem envia conjuntos de segmentos UDP com **TTL crescente** (1, 2, 3, ...).
- O n -ésimo roteador no caminho recebe o datagrama com TTL=0 → descarta e devolve **ICMP type 11, code 0** (TTL expirado), incluindo seu IP.
- Medindo o RTT de cada resposta, mapeia-se o caminho salto a salto.

Tabela 7: Principais Tipos e Códigos de Mensagens ICMP

Type	Code	Significado
3	0	Rede de destino inalcançável
3	1	Host de destino inalcançável
3	3	Porta inalcançável
8	0	Echo request (ping)
11	0	TTL expirado
12	0	Cabeçalho IP ruim

- **Parada:** quando o UDP finalmente chega ao destino, este responde **ICMP type 3, code 3** (porta inalcançável) → a origem para.

Amarração com a prova-base

Tabela 8: Mapeamento de Tópicos com a Prova-base

Questão	Como o Dia 1 resolve
Q1 (V/F) ICMP/OSPF/BGP no plano de controle	Seções 5.3 / 5.4 / 5.6 — todos pertencem ao plano de controle ✓
Q1 (V/F) 3 características do SDN	Tabela das características do SDN (Seção 5.5) ✓
Q1 (V/F) OSPF (estado de enlace) × RIP (vetor de distâncias, não “de caminhos”)	Seções 5.2/5.3 — atenção à pegadinha: <i>vetor de caminhos</i> é o BGP ✓
Q6 Estado de enlace × vetor de distâncias	Seção 5.2 completa, com ambos os exemplos numéricos ✓